

Understanding Memory Resources

Sure! Let's dive into the concept of **Spark Memory Management** in simple terms.

When you run a Spark application, it uses memory to perform tasks efficiently. Think of memory as a workspace where Spark can store and process data. Just like a chef needs enough counter space to prepare a meal without clutter, Spark needs enough memory to handle its operations smoothly. If the memory is too low, Spark might have to spill data onto the disk, which is like moving your ingredients to a smaller table when the counter is full—this slows things down!

In Spark, there are two main types of memory: **executor memory** and **storage memory**. Executor memory is used for processing tasks, while storage memory is used for caching data. Imagine executor memory as the area where you chop vegetables, and storage memory as the fridge where you keep your ingredients fresh. If you have too much in the fridge (storage memory), it can take away space from your chopping area (executor memory), making it harder to work efficiently. By managing these memory types wisely, you can ensure that your Spark application runs smoothly and quickly

Configuring Spark Process Memory

Spark applications allow configuration of driver and executor memory

Upper limit on useable memory enables apps to run without using all available cluster memory

Exceeding memory requirements causes disk spill or out-of-memory errors

- When running a Spark application, the driver and executor processes launch with an upper memory limit. The upper memory limit enables a Spark application to run without using all the available cluster memory, but this limit also requires that the memory limits are set high enough to perform necessary tasks. An application runs best when processes complete within the requested memory. If the driver and executor processes exceed the memory requirements, this situation can result in poor performance with data spilling to disk or even out-of-memory errors.

Memory Setting Considerations

Executor Memory:

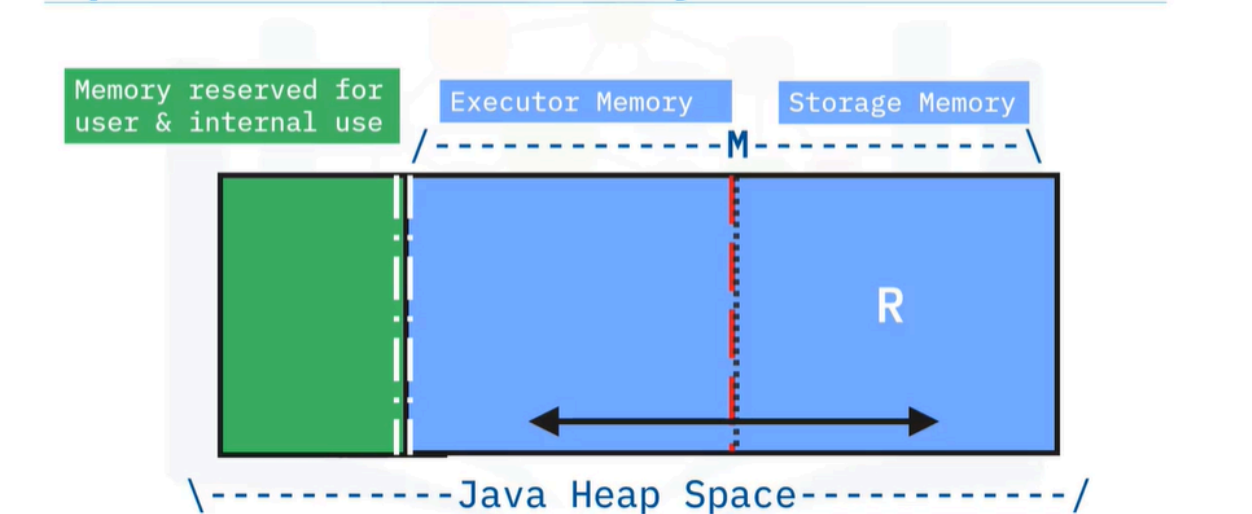
- Processing
- Caching
- Excessive caching leads to issues

Driver Memory:

- Loads data, broadcasts variables
- Handles results, such as collections

- Let's examine some memory setting considerations for both executors and drivers. Executors use memory for processing and additional memory if caching is enabled. However, excessive caching can lead to out-of-memory errors. Collecting data as a result of operations will be done in the driver. Driver memory loads the data, broadcasts variables, and handles results, such as collections. Because large data sets can easily exceed the driver's memory capacity, if collecting to the driver, filter the data and use a subset of the data.

Spark Unified Memory



- In Spark, executor memory and storage memory share a unified regions shown in this Java Heap Space in the space labeled M. When no executor memory is used, storage can acquire all the available memory and vice versa. Executor memory can evict storage memory if necessary, but only until total storage memory usage falls under a certain threshold.
- In other words, R describes a subregion within M where cached blocks are never evicted. Storage is not allowed to evict executor memory due to complexities in implementation. This design ensures several preferable properties.
- First, applications that do not use caching can use the entire space for executor memory, obviating unnecessary disk spills. Second, applications that do use caching can reserve a minimum storage space, the area labeled R, where their data blocks are immune to being evicted. Lastly, this approach provides reasonable out-of-the-box performance for a variety of workloads without requiring user expertise of how memory is divided internally.

Spark Data Persistence

Spark data persistence:

- Store intermediate calculations
- Persist to memory/disk
- Less computation
- Faster iterations over data



- Next, let's explore data persistence in Spark. Data persistence, or caching data, in Spark means being able to store intermediate calculations for reuse. By setting persistence in either or both memory and disk. After the intermediate data is calculated to produce a new DataFrame, and if memory is cached, then any other operations on that DataFrame can reuse the same data, rather than re-loading data from the source and re-calculating all prior operations. This capability is essential to speed machine learning workloads that often require many iterations over the same data set when training a model.

Data Persistence Example

Caching a DataFrame with a features column of random values

```
# Define DataFrame with feature column
df = spark.range(100).withColumn("features", rand()).cache()

# Features are generated then cached first time action is applied
print(df.filter(col("features") > 0.5).count())

# Cached DataFrame with features is reused in subsequent calls
print(df.filter(col("features") < 0.5).count())
```

- This sample PySpark code creates a DataFrame with column features consisting of random values. After creating the DataFrame, the 'cache()' method is called to mark the random values to cache in memory. Caching the DataFrame here means that the application only needs to generate random features once. At this point, the random values are not yet generated; caching is done lazily after computing the values. In this example, the count() action is invoked on the DataFrame, which generates the values that are then stored in memory. Subsequent DataFrame calls can use the cached values, which can save a great deal of computational time if the cost to recreate a DataFrame is high. If this DataFrame is not cached, then different random features would be generated with each action on the DataFrame, because the function rand() is called each time.

Setting Memory on Submit

Set the executor memory for a Spark standalone cluster:

```
$ ./bin/spark-submit \
--class org.apache.spark.examples.SparkPi \
--master spark://<spark-master-URL>:7077 \
--executor-memory 10G \
/path/to/examples.jar \
1000
```

Set each executor to use up to 10GB of memory

- There are several ways to set the memory for executors in a cluster, such as setting a value in the properties file. However, the more common practice is to specify a memory configuration when submitting the application to the cluster. You can tailor memory so that each application has enough memory to run effectively but does not use all available memory in the executor. This example configures the application to run on a Spark standalone cluster and reserves ten gibibytes per executor when running tasks.

Setting Worker Node Resources

Set the Spark Standalone worker memory and cores:

```
# Start standalone worker with MAX 10Gb memory, 8 cores
$ ./sbin/start-worker.sh \
  spark://<spark-master-URL> \
  --memory 10G --cores 8
```

Default

- All available memory minus 1GB
- All available cores

- If using the Spark Standalone cluster to manage and start a worker manually, you can specify the total memory and CPU cores that the application can use. These specifications determine the resources available when workers start the executors. Avoid assigning more resources than are available on the physical machine. For instance, if the machine has a CPU with eight cores and the worker starts with 16 cores, too many threads might run simultaneously and cause performance degradation. You can tailor memory so that each application has enough memory to run effectively but does not use all available memory in the executor. The default configuration is to use all available memory minus 1gigabyte for all available cores.

Summary

In this video, you learned that:

- Spark has configurable memory for executor and driver processes
- Executor memory and Storage memory share a region that can be tuned as needed
- Caching data can help improve application performance